Pioneer Console — plan from the GUI meeting

Twenty-one notes, grouped, costed and sequenced · 2026-08-12

How to read this

Every item below comes from the meeting notes. Each one says what it means, **what the code does today** (checked, not assumed), and what the work is.

Effort is **S** (an hour or two), **M** (half a day to a day), **L** (multi-day, or needs a decision first).

Decisions from the review are folded in and marked **Decided**. Two items are **parked** — C6 and D2 — for reasons given in place. LightGBM has been dropped: it is a Pioneer question, not a console one, and this document is GUI only.

A. Chrome and layout

A1. Collapse icon smaller, moved to the top — S

Today the collapse control is a full-width pill at the *bottom* of the sidebar with a chevron and the word "Collapse", `padding: 12`, `font 600 13.5px`. It is the heaviest element in the sidebar for something pressed once a session.

Move it to the top bar of the sidebar as an icon-only button, drop the label and the fill. This pairs with A2 — both free space at the top of the sidebar.

A2. Remove the large logo — S

`PioneerLogo` sits in its own block with a bottom border, and since the title-bar change it also carries the traffic-light clearance. Removing it reclaims roughly 90px at the top of the sidebar.

Worth deciding what, if anything, replaces it. Options: nothing (the window title and the Dock icon already identify the app), a small wordmark on the same row as the collapse button, or keep the peak mark at a much smaller size. A1 and A2 should be designed together as one header row.

B. Naming and wording

These are all one-line changes to labels and hints. Grouped because they should be reviewed together for tone, and shipped as one commit.

B1. "Calibration file" — S

The name says *how* it is used rather than *what to give it*. Its actual job is to supply a reference run from which the fragment and precursor m/z ranges are detected (`auto_detect_frag_bounds`).

Suggested: **Reference MS file**, hint *"Any run from this experiment — used to detect the fragment and precursor m/z range."*

B2. "NCE" — S

The field is a starting guess that Pioneer refines, not a fixed instrument setting. Suggested: **NCE (starting estimate)**, hint noting it is refined during the search.

B3. "Isotopes" — S

Ambiguous between precursor and fragment isotopes. The parameter is fragment isotopes; the label should say so.

B4. Debug logging level — S/M

A select for the log verbosity Pioneer emits. **Open question:** what levels does Pioneer actually accept, and is it a CLI flag, an env var, or a config key? The GUI work is trivial once that is settled; the answer decides whether this is S or M.

C. Paths and file selection

The largest cluster, and the one with the most existing machinery to build on.

C1. Library output accepts a folder that does not exist — S

Today `browseLibPath` calls `pickFolder`, so the native dialog can only return a directory that already exists; the `.poin` name is appended afterwards. That means you cannot name a new library in the place you want it.

Switch to a save dialog seeded with a default filename, and keep appending `.poin` when it is missing.

C2. Configurable default browse location — S (partly built)

Already done: `backend.ts` keeps `pioneerConsole.lastDir` in localStorage and passes it as the dialog's `defaultPath`, so pickers reopen where you last were, across sessions as well as within one.

Not done: there is no configured default and no home-directory fallback — with no stored value the dialog opens wherever the OS decides. Add a setting for a preferred root, and fall back to it, then to home, before letting the OS choose.

C3. Results folder derived from the MS data folder — S

A button beside *Results folder* that fills it from the MS data folder plus a suffix. **Open question:** what suffix — a fixed `_results`, the date, or the run name the GUI already generates? The run name would make results self-describing but changes on every run, so it cannot be filled in ahead of time without also pinning the name.

C4. Remember the last spectral library — S

The library path is the least-changing field in a SearchDIA run and the most annoying to re-pick. The draft is already persisted under `pioneerConsole.v2`, so the gap is that a fresh install starts empty rather than that the value is lost. Cheapest version: keep a most-recently-used list of libraries and offer it as a dropdown on the field.

C5. Drag a folder onto the browse bar — M

Tauri v2 gives file-drop events per window; the work is a drop target on each path field plus the hover affordance, and rejecting drops of the wrong kind (a file where a folder is wanted). Do it once as a shared component so all three forms get it.

C6. Pick a subset of files in the MS data folder — L

Parked — do this last.

Today the field is a folder and Pioneer takes everything in it. Selecting a subset needs a file list with checkboxes in the UI, and — the harder half — a way to express that subset to Pioneer. If SearchDIA only accepts a directory, the choices are a temporary folder of links or a change on the Julia side.

Because it may require a Pioneer-side change, it goes last: everything else here is contained within the console, and none of it should wait behind a cross-repo change.

D. Library awareness

D1. Show metadata from the library — M

Once a `.poin` is selected, read and display what it is: the prediction model, digest parameters, modifications, precursor count. This turns the path field from a string into something you can sanity-check before a two-hour run.

The library is a directory with an Arrow table plus config, so this is a Rust side read of the library's own `config.json` (the inspector already detects `.poin` layout via `PION_MARKERS`) and a small summary panel under the field.

Decided: read the library's config, not its filename. Filename parsing would be cheaper but breaks the moment a library is renamed or built elsewhere, and it cannot report anything the name does not already say.

D2. DownloadSpecLib as a fourth workflow tab — L

A new command alongside ConvertRAW / BuildSpecLib / SearchDIA for fetching a prebuilt library.

Parked — there is no download location yet.

The GUI half is well understood; it is the same form-and-run shape as the other three tabs. All of the risk is in the source and transport: what is being downloaded from, whether there is an index to list, and whether the transfer needs resume and checksum verification.

A mock-up can be built against a stub source whenever it is useful for reviewing the shape of the tab, but the real work waits for a location.

E. Run lifecycle

E1. A console window opens on Windows every run — S, real bug

`runner.rs` spawns with a plain `Command::new`. On Windows, a GUI process starting a console subsystem executable gets a console window unless `CREATE_NO_WINDOW` (`0x08000000`) is set through `std::os::windows::process::CommandExt::creation_flags`.

The `taskkill` call in the cancel path needs the same treatment, or cancelling flashes a second window.

Cheap and self-contained, but only verifiable on a Windows build.

E2. Open the results folder when a run completes — S

Needs `tauri-plugin-opener`; only `tauri-plugin-dialog` is in `Cargo.toml` today. Add the plugin, then a button on the completed-run view and in the log drawer's header.

Worth pairing with the existing check for whether the folder still exists — the drawer already reports when a restored run's output has been moved or deleted, and the button should be hidden or disabled in that case rather than opening nothing.

E3. Fewer default threads for BuildSpecLib — S, but answer the question first

Open question, and it should be measured rather than guessed: how much does thread count actually affect a library build? The phases are Koina requests (network-bound, governed by `max_koina_requests`, currently 24) and local fragment prediction and indexing (CPU-bound).

The 7.2-minute Prosit build I ran earlier spent 300s of its 431s in Fragment Prediction, which suggests threads do matter there. One build at 4 threads versus one at 15 would settle it in under half an hour, and the answer is worth knowing regardless of what the default becomes.

F. State and persistence

F1. Queue and history numbering — S, needs one clarification

Decided, and the two lists differ:

- **Queue** renumbers positionally. This is what `{idx + 1}` already does, so the queue needs no change — position in the queue is the useful fact.
- **History** takes the next number and keeps it. A monotonically increasing run counter that never resets and never renumbers when an entry is deleted, so the thousandth search is #1000 no matter what has been removed.
- **No cap on history.** The current `HISTORY_LIMIT = 100` goes.

The counter is stored with each run, which also makes F3 cheaper.

One caution on removing the cap: history lives in `localStorage` today, a low-single-digit-megabyte budget shared with the form draft. Parameters run 1–2 KB per entry, so a thousand runs is survivable, but the failure mode when it is not is a quota error that can take the whole store down. Until F2 lands, removing the cap needs a quota guard that trims the oldest entries on a write failure rather than losing everything. This is the strongest argument for doing F2 sooner rather than later.

F2. Move history and queue to SQLite — M/L

Today: `localStorage`, with history capped at 100 runs and parameters only — no log output, because a single SearchDIA log is megabytes against a low-single-digit-megabyte budget shared with the form draft.

SQLite removes the cap, makes F3 a query instead of a scan, and would let logs be kept. Needs `tauri-plugin-sql` or `rusqlite`, a schema, and a migration that imports whatever is in `localStorage` so existing history is not dropped.

Do this before F3 and F4 rather than after — both are much cheaper on top of a real store, and doing them first means writing them twice.

F3. Search the run history — S once F2 lands, M before it

A filter over run name, command, target path and date.

F4. Persist the queue across restarts — M

Two behaviours are possible and they are not the same: re-queue the pending runs on launch, or move them to history as "interrupted". Re-queuing risks starting a long job the moment the app opens; the note suggests you are open to the simpler option. My recommendation is to record them as interrupted and offer a one-click re-run, which is safe by default and loses nothing.

Suggested sequencing

First — the cheap wins and the real bugs. All small, all independent, all visible immediately:

- E1 Windows console window (a bug, and it makes the app look broken)
- C1 library output save dialog (a bug, blocks a normal workflow)
- B1–B3 the renames
- A1 + A2 sidebar header, designed as one change
- E2 open results folder

Second — the path work, which shares a component and should not be spread out: C2 configured default, C3 derived results folder, C4 remembered library, C5 drag and drop.

Third — F2 SQLite, then F1, F3 and F4 on top of it. Doing F3 or F4 before the store means writing them twice.

Fourth — D1 library metadata, now that reading the config is settled.

Parked, in this order when they unpark — D2 DownloadSpecLib when there is a location to download from, and C6 file subset last of all, since it is the only item that may need a change in Pioneer itself.

In parallel, whenever — E3 is a measurement, not a feature. One build at 4 threads against one at 15 answers it in half an hour, and the answer is worth having whatever the default becomes.

Open questions, collected

Answered in review: **D1** (read the config), **F1** (queue positional, history monotonic and uncapped), **C6** and **D2** (parked).

Still open:

1. **B4** — what logging levels does Pioneer accept, and how are they set?
2. **C3** — what suffix should the derived results folder use?
3. **F4** — re-queue interrupted runs, or record them as interrupted? My recommendation is to record them, since re-queuing risks starting a long job the moment the app opens.
4. **C6** — whether SearchDIA can take an explicit file list. Not needed until C6 unparks, but the answer decides how big it is.